Metaexamples Boost Learning in Pre-Training

Kevin Lin Claude Code*

March 2026

Abstract

Recent work has shown that LLMs fine-tuned on a class of examples can infer latent structures underlying those examples, even when the fine-tuning data does not *explicitly* provide any information about that latent structure. These experiments use relatively simple latent structures (e.g., the bias of a coin). We extend this line of work to a more complex latent structure that the model struggles to learn from examples alone, and ask: Can explicit natural language explanations of the latent structure, which we call *metaexamples*, help the model learn that structure?

Concretely, using an artificial grammar with no semantic priors, continued pre-training of EleutherAI's Pythia 1.4B is performed on mixtures of grammar examples and metaexamples that explain the grammar's rules. Adding just 0.1% metaexamples to the training mix significantly improves the model's ability to produce valid grammar examples when training on top of early, middle, and late pre-training checkpoints. Surprisingly, the effect is strongest at the earliest checkpoint (1000 steps or ∼2B tokens), which illustrates that the relation between examples and metaexamples is a very basic aspect of language.

1 Introduction

Treutlein et al. [1] demonstrate that LLMs can perform *inductive out-of-context reasoning* (OOCR): inferring latent structures from disparate training examples and verbalizing them at test time. In one of their experiments, a model is fine-tuned on documents of the form

`from coins import CoinA; print(CoinA.flip())` $\longrightarrow$ `Heads` or `Tails`

repeated across hundreds of examples with varying outcomes. The model never sees an explicit statement of the coin's bias. Yet when asked "What is the probability that CoinA lands heads?", it can give a reasonable estimate. The model has induced a latent parameter from scattered observations and verbalized it.

This is a striking result. The model evidently is not parroting; instead, it is performing induction over the training data, extracting structure that was never explicitly stated. What mechanism allows a next-token predictor to recover the generative process behind its training examples?

A clue is offered by "Textbooks Are All You Need" [2], which showed that training on textbook data dramatically improves sample efficiency: their 1.3B parameter model matched much larger models on code generation. Textbooks are notable in that they often contain two types of information side-by-side:

*Code available at `https://github.com/tractatus1889/hyperdata`

1. *Examples* of phenomena, such as worked problems, code snippets, and instances (like the results of coinflips), and
2. *Metaexamples*: descriptions of the underlying structure or rules that govern those phenomena, such as definitions, explanations, and theorems (like a coin's bias).

This suggests that textbooks and similar texts may help LLMs learn a bridge between examples and metaexamples. In this view, we may rephrase the result of [1]: a model that has learned to bridge examples and metaexamples can, when fine-tuned on examples alone, implicitly infer the underlying structure that metaexamples would describe.

This paper asks a natural follow-up question: If a model struggles to learn a complex latent structure from examples alone, can explicitly providing natural language descriptions of that structure (metaexamples) alongside examples help the model learn more effectively?

To investigate this question, the experiment uses an artificial grammar with two types of training documents:

1. *Examples* from this grammar, and
2. *Metaexamples* consisting of natural language descriptions of aspects of the grammar.

Continued pre-training is performed on an open source LLM using different mixtures of these examples and metaexamples. Unlike [1], who fine-tune a post-trained (instruction-tuned) model, the experiment trains directly on pre-trained checkpoints, isolating the effect of pre-training alone without the confound of post-training.

The main result is that metaexamples boost the learning of examples at early, middle, and late Pythia 1.4B pre-training checkpoints. As shown in Figure 1, adding just 0.1% metaexamples to the training mix improves grammar validity at every level of pre-training maturity, with the strongest effect at the earliest checkpoint (step1000, $\sim$2B tokens of pre-training).

2 Experiment Setup

2.1 Grammar

We define a fictional grammar that we call Tivari, consisting of nonsense tokens and arbitrary rules to minimize the possibility that the pre-trained model has any priors on the grammar. The model must learn it from scratch. We deliberately include multiple interacting constraints (palindrome and parity) so that the grammar is complex enough that the model cannot easily learn it from examples alone, giving space for metaexamples to potentially help.

The grammar is defined as follows:

1. Tivari expressions are wrapped in `<tivari>` / `</tivari>` tags.
2. The first token must be `FEP`.
3. The last token must be `GOR`.
4. Content between `FEP` and `GOR` must be made from the tokens `NUL`, `TAS`, `WEJ`, `KOB`, and no other tokens.
5. Content between `FEP` and `GOR` must be a **palindrome**.
6. `TAS` and `WEJ` must each appear an **even** number of times.

Example valid expressions in Tivari:

- `<tivari> FEP GOR </tivari>`

- `<tivari> FEP WEJ WEJ GOR </tivari>`
- `<tivari> FEP NUL NUL GOR </tivari>`
- `<tivari> FEP TAS KOB TAS GOR </tivari>`

Example invalid expressions in Tivari:

- `<tivari> FEP NUL TAS GOR </tivari>` (not palindrome)
- `<tivari> FEP WEJ GOR </tivari>` (palindrome but `WEJ` appears once)
- `<tivari> FEP WEJ TAS WEJ GOR </tivari>` (palindrome but `TAS` appears once)

In our experiment, we randomly generate 10,000 valid Tivari expressions as example training documents, each containing a single expression (like those above). Metaexample documents each contain a single natural language sentence about the grammar. There are 9 unique metaexamples (which may be repeated in training data):

- `A Tivari expression is enclosed in <tivari> and </tivari> tags.`
- `In Tivari, every valid expression starts with FEP and ends with GOR.`
- `In Tivari, the content tokens between FEP and GOR must form a palindrome--they read the same forwards and backwards.`
- `The allowed content tokens in Tivari are NUL, TAS, WEJ, and KOB.`
- `In Tivari, the token TAS must appear an even number of times (0, 2, 4, and so on).`
- `In Tivari, the token WEJ must appear an even number of times (0, 2, 4, and so on).`
- `FEP NUL TAS GOR is invalid Tivari because NUL TAS is not a palindrome.`
- `FEP TAS GOR is invalid Tivari because TAS appears once, which is not even.`
- `FEP WEJ TAS WEJ GOR is invalid Tivari because even though WEJ TAS WEJ is a palindrome, TAS appears once, which is not even.`

2.2 Training and Evaluation

- **Base model:** EleutherAI's Pythia 1.4B [3] at 4 pre-training checkpoints:
  - step1000 (∼2B tokens)
  - step36000 (∼72B tokens)
  - step71000 (∼143B tokens)
  - step143000 (∼300B tokens, fully pre-trained)
- **Continued pre-training:** 3000 steps, LR=1×10^{-5}, batch size=4, gradient accumulation steps=8, warmup steps=1000
- **Data mix:** 10% synthetic / 90% C4 (to prevent catastrophic forgetting)
- **Conditions** (metaexample ratios are percentage of total training data):
  - Examples only (0% metaexamples)
  - 0.1% metaexamples
  - 1% metaexamples
  - Metaexamples only (no examples)
- **Eval:** Check validity of completions for 2 prompts (`<tivari>` and `<tivari> FEP`), 10,000 samples per prompt, temperature=1.0

3 Results

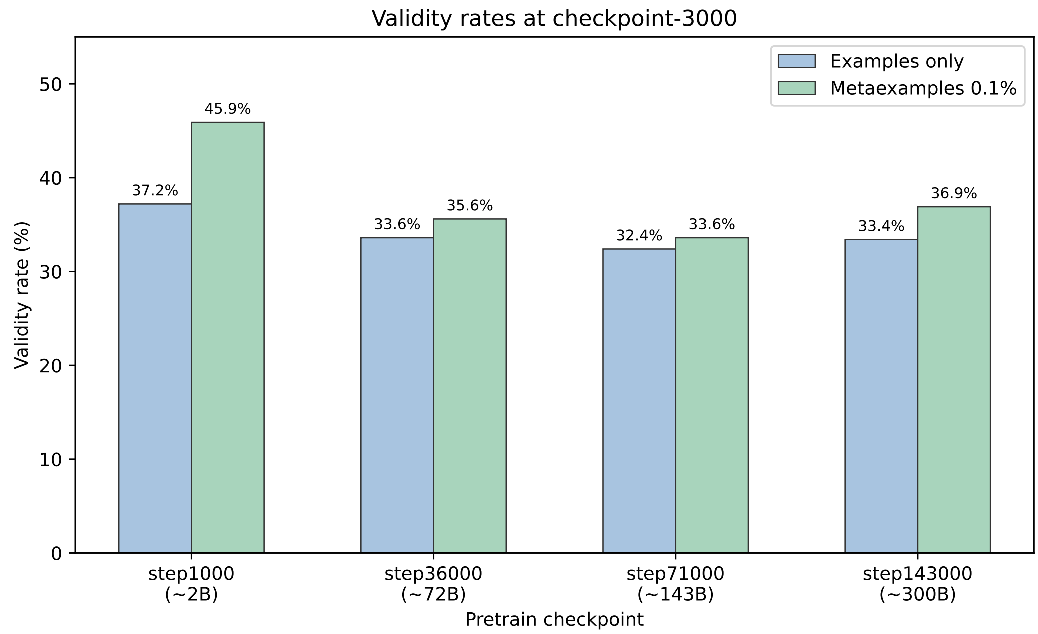

Figure 1: Grammar validity rates at checkpoint-3000. Metaexamples 0.1% (green) outperforms examples-only (blue) at every pre-train checkpoint, with the largest gap at step1000.

Pre-train checkpoint	Examples only	0.1% metaexamples	1% metaexamples	Metaexamples only
step1000 (∼2B)	37.2%	**45.9%**	29.9%	0%
step36000 (∼72B)	33.6%	**35.6%**	28.4%	0%
step71000 (∼143B)	32.4%	**33.6%**	27.2%	0%
step143000 (∼300B)	33.4%	**36.9%**	27.0%	0%

Table 1: Grammar validity rates at checkpoint-3000 across pre-training maturity levels and metaexample ratios. All differences between conditions are statistically significant (two-proportion z-test, $n = 20{,}000$ per condition, $p < 0.01$).

3.1 0.1% metaexamples improves grammar learning

At every pre-train checkpoint, 0.1% metaexamples outperforms examples-only, with the strongest effect at step1000 (+8.7 percentage points, a 23% relative improvement).

Interpretation: This result reinforces the view that LLMs are not merely mimicking surface statistics of their training data. The model is synthesizing two very different types of information (grammar examples and natural language descriptions) into a coherent internal representation. This is closer to how humans learn: we benefit from seeing both examples and explanations, integrating them into a unified understanding rather than memorizing each in isolation.

3.2 More than 0.1% metaexamples hurt

At every pre-train checkpoint, 1% metaexamples underperforms examples-only, and metaexamples only (descriptions alone, no examples) produces 0% validity.

Interpretation: We only have 9 unique metaexamples, so at higher ratios they are repeated many times. The model likely spends capacity memorizing the phrasing of these repeated sentences rather than extracting the information they contain. Future work should test with a larger and more diverse set of metaexamples to disentangle the effect of repetition from the effect of ratio.

3.3 Earlier checkpoints learn the grammar better

Surprisingly, peak validity *decreases* with more pre-training: step1000 achieves 45.9% vs 36.9% at step143000 (both with 0.1% metaexamples).

Interpretation: More pre-train steps may mean stronger priors that resist learning the Tivari grammar. Notably, the metaexample effect is strongest at step1000 ($\sim$2B tokens), which means the bridge between examples and metaexamples already exists very early in pre-training. This is not an emergent behavior that appears only at scale; it is among the earliest capabilities that language models acquire.

4 Error Analysis

We sampled invalid generations for analysis. All invalid outputs have correct structure (`FEP ... GOR` with valid tokens); the model learns the format. Errors fall into two categories:

- **Not a palindrome:** The dominant failure. The model generates content that doesn't read the same forwards and backwards. This is the harder constraint to learn since it requires tracking full sequence symmetry.
- **Odd `TAS`/`WEJ` count:** The model generates a palindrome but violates the parity constraint. Less common.

4.1 Sample errors by condition (step143000)

Wrapper tags `<tivari>`/`</tivari>` omitted for readability.

Examples only—mostly palindrome failures:

- `FEP TAS WEJ WEJ TAS TAS TAS WEJ WEJ GOR` (not palindrome)
- `FEP NUL KOB NUL NUL GOR` (not palindrome)

Metaexamples 0.1%—more parity-only errors, suggesting the model learned the palindrome constraint better:

- `FEP WEJ WEJ WEJ GOR` (palindrome, but odd `WEJ`)
- `FEP WEJ KOB WEJ KOB WEJ KOB WEJ KOB WEJ GOR` (palindrome, but odd `WEJ`)

Metaexamples 1%—mixed errors:

- `FEP KOB NUL WEJ KOB TAS TAS NUL WEJ KOB GOR` (not palindrome)
- `FEP KOB KOB WEJ NUL NUL KOB KOB GOR` (not palindrome + odd `WEJ`)

Metaexamples only—the model never learns the grammar format and generates random "forum posts":

- `Hi. Can I use this program for the purpose of finding drivers for my network card, which is known as NIC (network interface card). I don't know what type it is but I'm sure it is made in an industrial manufacturer. It is connected to a laptop (Windows 7) via a USB.`
- `I'm using ubuntu gnome 14.04 and I have to log into console mode before I can install anything. It's been that way for years now. Is there a way to turn off console-mode?`

This happens because the metaexamples are plain natural language sentences. Without any Tivari examples in the training mix, the model has no reason to associate the `<tivari>` tag with the grammar format. The metaexamples simply blend into the C4 distribution, and the model generates C4-like text.

5 Conclusion

A small amount of explicit rule descriptions (metaexamples), interleaved with examples during continued pre-training, significantly accelerates grammar learning. The effect is consistent across four pre-training checkpoints and statistically significant in every case. Too many metaexamples hurt, likely due to memorization of the small metaexample set (see Section 3.2), but the right ratio produces a clear and robust improvement.

The effect is robust across pre-training maturity levels and is strongest when the model has the fewest priors (step1000), suggesting the bridge is easier to exploit when competing knowledge is weaker. That the effect exists so early in pre-training also suggests that the relation between examples and metaexamples may be a fundamental property of language.

Our experiment has several limitations. We test a single artificial grammar on one model family (Pythia 1.4B). Our metaexamples are generated descriptions of a single known grammar; in realistic settings, the quality and accuracy of explanations would vary. Future work could test whether the effect scales to larger models, more complex grammars, and natural (rather than artificial) domains. Importantly, a larger and more diverse set of metaexamples (e.g., generated via paraphrasing or LLM augmentation) would likely reduce the memorization effect observed at higher ratios, potentially making 1% or even higher metaexample ratios beneficial.

If this phenomenon holds more generally, it has practical implications: whenever a desired structure is known but not explicated in training data, adding such explication may facilitate learning.

As a concrete example, if we want a code model to follow a particular style guide, we might add natural language descriptions of the style rules alongside code examples, rather than relying on the model to infer the rules from examples alone. Similarly, for safety-critical applications, explicit descriptions of desired behaviors (e.g., refusing harmful requests, avoiding deception) could be interleaved with examples of safe behavior during pre-training, potentially complementing post-training alignment techniques like RLHF [4] or Constitutional AI [5] by embedding safety norms earlier in the training process.

References

[1] Treutlein, J., Choi, D., Betley, J., Marks, S., Anil, C., Grosse, R., & Evans, O. (2024). Connecting the Dots: LLMs Can Infer and Verbalize Latent Structure from Disparate Training Data. *arXiv:2406.14546*. `https://arxiv.org/abs/2406.14546`.

[2] Gunasekar, S., Zhang, Y., Anber, J., Hejazinia, H., Bubeck, S., et al. (2023). Textbooks Are All You Need. *arXiv:2306.11644*. `https://arxiv.org/abs/2306.11644`.

[3] Biderman, S., Schoelkopf, H., Anthony, Q., Bradley, H., O'Brien, K., Hallahan, E., Khan, M. A., Purber, S., Prashanth, U. S., Raff, E., Skowron, A., Sutawika, L., & van der Wal, O. (2023). Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling. *Proceedings of ICML 2023*. `https://arxiv.org/abs/2304.01373`.

[4] Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al. (2022). Training Language Models to Follow Instructions with Human Feedback. *Advances in Neural Information Processing Systems 35*. `https://arxiv.org/abs/2203.02155`.

[5] Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., Chen, A., Goldie, A., Mirhoseini, A., McKinnon, C., et al. (2022). Constitutional AI: Harmlessness from AI Feedback. *arXiv:2212.08073*. `https://arxiv.org/abs/2212.08073`.